SQL

Constraints :-

They are Rules and Regulations which we have to follow when we are Entring in any tables.

There are Six Types of Constraints.

1. Not Null
2. UNIQUE
3. Default
4. Primary key
5. foreign key
6. ~~Unique key~~ CHECK

Not Null

जिस भी column के आगे Not Null होगा हम उस column को खाली नहीं रख सकते।

i.e

```
Create table Employee
(
    id int NotNull,
    Name Varchar(100) NotNull
    Age int
)
```

अब हम id और Name में data insert करना mandatory होगा। अगर हम Employee table में id और Name में कोई value नहीं देंगे तो यह error generate करेगा।

② UNIQUE :- No-duplication.

यह किसी column पर लगाते हैं उस column की value को किसी दूसरे column में repeat नहीं किया जा सकता।

i.e Creat tabl Employee
(
id int unique,
Name varchar(50) unique,
Age int
)

यहाँ id अगर 1 है तो यह 1 अगले Repeat नहीं होगा केवल unique वाले column पर अगर Name = abc है तो यह अगले Repeat नहीं होगा।

Note: जब किसी column पर Primary key लगाते हैं तो Not Null + unique दोनों काम होते हैं।

① एक Table में unique key multiple हो सकती है
② duplicacy नहीं होगी
③ 1 बार Null हो सकती है
④ Non-Clustered index are used in unique key.

Default

Set a default value for a column when No value is specified.
मैं एक ऐसा column रखता हूँ जिसमें अगर user खाली छोड़ दे। तो उसमें default value insert हो जाये

i.e.
```
Create table Employee (
    id int primary key identity,
    Name varchar(100),
    Voter_Age int default(18),
    Salary int
)
```

अब अगर user voter_Age column को छोड़ देगा तो उसमें default value 18 हो जायेगी।

CHECK

यह Check करेगा कि कोई value check constraint को satisfied करती है या नहीं अगर नहीं करेगी तो data insert नहीं होगा error आयेगी।

i.e.
```
Create Table Employee
(
    id int primary key identity,
    name varchar(100),
    voter_Age int Check(voter_Age >= 18),
    Salary int
)
```

अगर voter_Age ज्यादा या equal है 18 के तब आगे काम होगा वरना नहीं।

Primary Key :-

जब किसी column पर primary key लगाते हैं तो Not Null + Unique दोनों property by default लग जाती हैं।

Primary key data को uniquely identify करती है।

- किसी भी Table में केवल 1 Primary key हो सकती है।
- इस column पर primary key use होती है Null and duplicate नहीं होती
- Primary key लगने पर clustered index by default लग जाता है

Foreign Key

- इस Table में multiple हो सकती है
- duplicacy Possible
- Null भी हो सकती है
- No - index use.

if

```
Create table Employee (
id int primary key identity,
Name varchar (100),
age int,
Order No int ; Foreign key Reference tall_Name (column)
```

Remove Constraints

Alter table table_Name Drop Constraint Constraint_Name.

Creating Foreign key in Existing table.

ALter table tabl_Name ADD Foreign key (C_id) Refference table_Name (other table) (C_id).

ALter table table_Name Add Unique (column_Name)

Collation:- Collation is deffined as a set of Rules That determine How data can be ~~Stored~~ Sorted as well as compared.

Different type of collation sensitivity are as follow.

- Case Sensitivity.
- Kana "
- width "
- Accent "

View

View is Virtual table. Just like a Table Not a table.

It's Create for Select Statment.

Syntax

Create View view-name
as
Select Statements ----.

We Can Create Stored Procedure On view
We Can Create Instead of Trigger on view

Note We Can't Create for trigger on view.

View is Virtual Table.

S.P. is ~~made some modific~~ (4) could modification on table.

View	Stored Procedure
① Not Accept Parameter	① Accept Parameter
② Can Contain Only Single Select Statment	② Contain Several Statment
③ Syntax Create view view_name as ~~begin~~ select query	③ Syntax Create Procedure Proced_Name as begin SQL command end
④ Can not Perform modification to any table	④ Can Perform modification on table
⑤ Select query से get कर सकते है।	⑤ Select query से get नहीं कर सकते।
⑥ faster as compare to Stored Proced	⑥ Slower.

View	table
① It is virtual table	① It is actual table
② delete view and Re-Create it, No data Loss.	② If delete table, data will be Loss..
③ depend on table	③ Independent.
④ Syntax difference Create view view-name as	④ Create Table TableName ( )
⑤ does Not occupies Space on the System	⑤ It occupies space on the system.
⑥ use to extract the data	⑥ use to store the ~~Result~~ data.
⑦ generate ~~fast~~ Slow Result	⑦ generate fast result.

Stored Procedure

we can write query inside it, and use it again and again

SYNTAX

```
Create  Proc  Procedure_Name ;
as
begin
   SQL - Command . . .
end
```

Types

① System Stored Procedure
② User - defined " "

Procedure with 1 Parameter

```
Create  Procedure  Procedure_Name
   @id int
as  begin
   Select * from table_Name where Colm_Name
   = @id
end
```

SP_helptext Procedure_name

Using Encryption in Stored Procedure

यदि कोई Procedure का Code कोई other User नहीं देख पाये।

i.e

```
Create Proc Procedure_Name
@id int , @name varchar(100)
with encryption
as
begin
Select * from table_name where
Colum nam = @id And Column_Name = @name
end
```

अगर हमें Encryption हटाना है तो
हम Procedure को ALter करके हटा देंगे।

Procedure with Try and Catch Block

function

- It's collection of SQL command we can use it again and again.
- ONLY work with Select Statement.
- Must Return a value.
- ONLY work with Input Parameter.
- Try and Catch Statment Not used in function.

There are two types of function..

① User-defined function

② System " "

USER-defined function

① Scalar function
② INLine table value function
③ multiple Statment table value function.

Syntax

```
Create function function-Name()
return int
as
begin
    SQL Statment
end
```

ONLY Return Single Value.
-take one or more parameter.
Value we be string and int.

without Parameter

```
Create function function_name()
Returns varchar(100)
as
begin
  Print 'Hello world';
end
```

[Select dbo.function_name();]

↓

find the @ function using this.

MULTIPLE Parameter

```
Create function function_name(@num as int,
@num1 as int)
Return int
as
begin
    return (@num + @num1)
end

Select dbo.function_name(3,5)
```

Scalar function

ONLY Return Single Value.
take one or more parameter.
Value me be String and int.

without Parameter

```
Create function function_name()
Return varchar(100)
as
begin
  Print 'Hello world';
end
```

[Select dbo.function_name();]

↓

find the @ function using this.

multiple Parameter

```
Create function function_name(@num as int,
@num1 as int)
Return int
as
begin
    return (@num + @num1)
end

Select dbo.function_Name(3,5)
```

INLINE Table Value function

- Return a table.
- No Begin end Block.

i.e

```
Create function function_Name ( )
Return table
as
return (select * from table_Name)
```

Multiple parameters

i.e

```
Create function function_Name (
@id int , @name varchar (100) )
Return table
as
Return (Select * from table_name where
    id = @id And Name = @name. )
```

Multiline table value function

- Return table with structure.
- include begin end block.

```
Create function function_name()
Return @table table_name (id int,
Name varchar(100), Age int)
as
begin
insert into @table
Select * from table_name
return
end
```

Inline Table value	multiline
① Return Clause can not Contain the Structure of table	① Contain the Structure of table.
② No Begin end block	② begin end block includ
③ batter in Performance as Compared to multi Line table value function	③ No Performance Advantage Here.
④ It work like view	④ It work like Stored Procedure.

Note: Local AND GLOBAL VARIABLE.

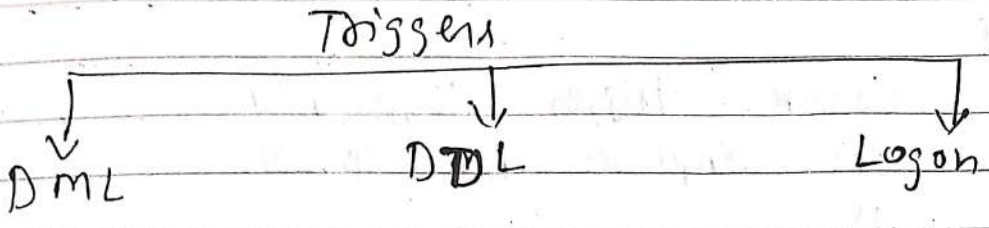
Local Variable :- these Variable can be used only INside the FUNCTION. It's Not used any other function.

GLOBAL VARIBLE :- they can be accessed throughout the Program. GLOBal Variable can Not be created whenever that function called.

Procedure	function
① It Return or may Not be Return	① Always Return a value.
② ONLy Integer Return	② ANy value Return
③ Try, Catch Block use	③ Not use Try; Catch Block
④ use Input and O/P Parameter	④ ONLy Input Parameter is use.
⑤ Can't be Called by a function	⑤ Can be Called by Stored Procedure.
⑥ Can't Call by SQL query	⑥ Can Call by SQL query ii Select * from dbo.fn();
⑦ we can use DML Command inside Procedures	⑦ Can't use DML Command

Triggers

It's different type of Stored Procedure.
It's Automatically invoke, when a Specical Event occur in db.



three types of triggers
1. DML
2. DDL
3. Log on.

DML Trigger

- यह DML Command Line पर चलेगा
Two types
1. for or after Trigger
2. Instead of trigger

FOR Trigger

Create trigger trigger_name
ON tabl_Name for insert
as
begin
--- SQL query ------
end

table_Name में जब भी insert होगा यह trigger चलेगा

जब भी Table-Name वाली की Table में insert होगा तो वह inserted वाली की Table में store हो जाएगा जो की एक magic table होगी।

i.e

```
Create trigger trigger-name
ON Employee for insert
as
begin
Select * from inserted  → magic table.
end
```

यह inserted (Magic Table) में store हुआ data latest होगा।

Note Magic table 2 होती है

① inserted
② deleted.

जब हम delet के लिए trigger बनायेंगे तो उसका data deleted में store होगा।

i.e

```
Create Trigger Tr-name
ON table-name for delete
as
begin
Select * from deleted
                Magic table.
end
```

Instead of Trigger

```
Create Trigger Tr-Name
on table-name instead of insert
as
begin
    Print 'Not Insert';
end
```

जब भी Instead of Trigger किसी Table पर लगाते है तो उस Table में insert, Delete, Update नहीं होगा क्योंकि begin or end के बीच SQL Statements, or message Print होता।

Other words : User Action नहीं चलेगा trigger Action चलेगा।

DDL TRIGGER

```
Create trigger trigger-Name
on database for Create-Table
as
begin
    Print 'New table Create';
end
```

जब Table Create होगी। यह Trigger Fire होगा

DDL Triggers Not Support Instead of Triggers. इसलिए, हम Rollback use करेंगे।

i.e

Create trigger tr_Name
On database for Create_Procedure.
As
begin
ROLLBACK
end

यहाँ Rollback की command Stored Procedure में create नहीं.. बल्कि होगी

Note SP_helptext trigger_Name

Logon Triggers

यहाँ हम set करेंगे कि कितने No. of user हमारे database को access कर सकेंगे हैं

Composite and Candidate Key

Composite Key →

A key which has multiple attribute to uniquely identify row in a table.

एक से ज्यादा Column को मिलाकर इस Table की Row के Data को identify करेंगे। Composite Key कहलायेगी।

Candidate Key :

Collection of Column (Uniquely identify Row data) जैसे (Aadhar No, Pen No, email id etc) are Candidate Key

एक Table में एक से ज्यादा ऐसे Colum हैं जिनसे इस Table के Row को identify कर सकते हैं। Candidate Key कहलाती हैं।

Title → Candidate Key

id	Pen-No	email	Aadhar No	Phon-No.	Name

Primary Key Candidate Key से ही चुनते हैं।

Transactions

सारी Command एक साथ Success होगी या एक साथ fail होगी।

यह एक group of SQL Command होता है।

Syntax

```
begin Transaction
SQL Statement - - - -
"   "  - - - - -
"   "  - - - - - -
~~End Trans~~.
Commite transaction
Rollback Transaction
```

Need क्या है।

इस में Record table में update करें। बाद में यदि आपने की पहले वाला Record सही था। तो हम इसके पास Undo का option नहीं होता। इसलिए इस Transaction use करते हैं।

Transaction can be Commite or RollBack.

ACID PROPERTY

Atomicity → सारी Command चलेगी / या सारी fail हो जायेगी। It manage by Transaction manager.

Consistency :- before Transaction and After Transaction data Consistency होनी चाहिए।

Isolation :- हर Transaction को अलग - 2 Execute करेगा।

Durability :- अगर कोई Transaction की हो तो उसको Commit या Rollback करना होगा।

Transaction with Try and Catch block

```
begin Try
begin Transaction
insert into Studen values ('A', 'B')
insert into Studen values ('A', 'B')
!
- - - - - - - - -
commit transaction
end Try
begin Catch
RollBack Transaction
    Print 'Transaction failed';
end Catch
```

Indexes

- Database से data को fast Retrieve करने के लिए Indexes use करेंगे।
- Indexes are Create on table and view
- अगर Table में Indexes नहीं है तो Query engine Table की हर Row को 1 by 1 check करेगा।
- Index जब भी Table पर लगायेंगे तो यह Table के data को Ascending or descding order में Sort करेगा।

Index Types

Indexes 2 Type की होती है

1. Clustered Index
2. Non-Clustered Index.

जब भी Primary key लगायेंगे Table में तब Clustered Index by default लग जायेगा Table में।

Clustered Index

Syntax

Create Clustered Index Index_Name
ON Table_Name (column_name desc)

Check for Index Apply on table.

SP_ helpIndex Table_Name;

DROP Index

drop Index Table_Name.IndexName

- Clustered एक Table में एक बार ही लगता है और यह same table में ही Sorting Apply करे देता।
- अगर हम Clustered Index delete करेंगे तो Primary key भी delete हो जाएगी।

Ex. Create Clustered Index index_name
ON Table_Name (Gender desc, Age ASC)

इस तरह से Index लगाते हो Composite Index कहते है।

NON - Clustered Index

- data store in 1 Place and index store in other Place
- जब भी इस Table पर NON-Clustered index लगाएंगे तो वह Table से अलग लगेगा। और वही data sort करेगा।
- एक Table में एक से ज्यादा Column पर NON - Clustered index लग सकते हैं

Syntax

Create index index_Name
ON table_Name (Cname ASC)

e.g

id	Name	Age	Salary
1	B	20	25000
2	C	25	22000
3	A	22	13000

after Applying index

Name	Row Address
A	Row Address
B	"
C	"

Clustered Index

① Primary key बनाते ही Clustered Index Automatically बन जाता है।

② Clustered Index Table में केवल 1 ही होता है।

③ जब भी लगेगा Table में ही Sorting Apply करेगा।

④ Syntax

Create Clustered Index Index Name ON table Name (Column Name ASC)

Non-Clustered Index

① ऐसा कुछ नहीं होता।

② Multiple Index होते है।

③ जब भी लगेगा Table से अलग Location बनाएगा और वहां Sorting करेगा।

Syntax

Create Index Index Name ON table_Name (Column_Name DESC)

UNION AND UNION ALL

2 से ज्यादा Table को merge करने के लिए UNION And UNION ALL का use करते है।

अगर हम दो SQL query को एक साथ Execute करते है तो उसका Result आएगा - 2 Result Set में आयेगा।

Ex Select * from table 1

Select * from table 2

O/P

id	Name	Age
1	A	23
2	B	24

id	Name	Age	Sele
1	A	21	10k
2	B	22	20k
3	C	23	30k

अब इन दोनो Table को O/P में एक ही Result Set में दिखाने के लिए हम Union or UNION ALL use करेंगे।
पर दोनो Table का data type और Colum Same होना चाहिए।

UNION

- यह duplicate Remove करता है।
- ONly Common data Collect करता है और Sorting Apply करता है।
- Select * from Table 1
  UNION
  Select * from Table-2

id	Name	Age
1	A	23
2	B	24

Table 1

id	Name	Age
1	C	21
2	D	25

Table-2

O/P

id	Name	Age
1	A	23
1	C	21
2	B	24
2	D	25

UNION के लिए दोनों Table में Same Column और same data type का होना चाहिए।

UNION ALL

यह duplicate ALLow करता है और Sort भी नहीं करता

```
Select * from table 1
UNION ALL
Select * from table 2
```

id	Name	Age
1	A	23
2	B	22
3	C	22

Table-1

id	Name	Age
1	B	21
2	E	25
3	A	26

O/P

id	Name	Age
1	A	23
2	B	22
3	C	22
1	B	21
2	E	25
3	A	26

JOINS

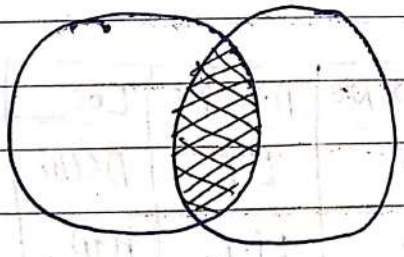
give o/p a singl table.

we can Apply Join on 2 or more than 2 tables.

Types of Join

1. INNER JOIN
2. Outer Join
   - → Left outer Join
   - → Right " "
   - → Full " "
3. CROSS JOIN.

INNER JOIN



id	Name	Gender
1	A	M
2	B	F
3	C	F
4	D	M
Employee		
5	E	M

id	Addrss	Salary	UId
1	ABC	22000	3
2	DEF	25000	4
3	GHI	23000	2
4	JKL	20000	1

details

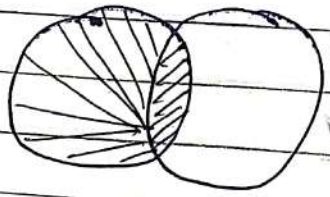
Select Name, Gender, Address, Salary from Employee inner Join details on Employee.id = details.UId.

O/P

Name	Gender	Address	Salary
A	M	ABC	22000
B	F	DEF	25000
C	F	GHI	23000
D	M	JKL	20000

Left JOIN

Return ALL ROW from Left and match value from Right table.



Emp No.	Emp Name	Dept No.
E1	Varun	D1
E2	Amit	D2
E3	Sumit	D1
E4	Nitin	

emp

Dept.No	D.Name	Loc
D1	IT	Delhi
D2	H.R	HYD
D3	FINANCE	PUNE

Dept

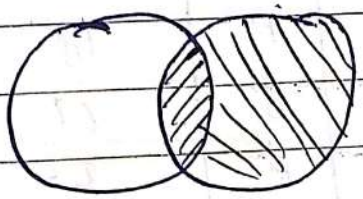
Select * Emp.No., EMP_Name, Dept.No, D.Name, Loc from emp Left Join Dept on emp.deptNo = Dept.Dept.No.

O/P

Emp No.	Emp Name	Dept No	D Name	Loc
E_1	Varun	D_1	IT	Delhi
E_2	Amit	D_2	H.R.	Hyd
E_3	Sumit	D_1	ANAME	PUNE.
E_4	Nitin	—	—	—

RIGHT OUTER JOIN

Return ALL Row. from Right and match value from left table.



~~Select * from~~

O/P

id	Name	Age	Salary
1	Avinash	22	22K
2	Sushan	25	25k
3	—	—	25K.

CROSS JOIN

Product

Pid	Name	Amount
1	Mobile	20,000
2	Laptop	40,000
3	TV	30,000

Customers

CName	Age	Loc
A	18	Noida
B	22	delhi
C	21	GHZ

Select Name, Amount, CName, Age

from Product, Customers

Name	Amount	CName	Age
1	Mobile	20,000	A	18
2	Laptop	40,000	A	18
3	TV	30,000	A	18
1	Mobile	20,000	B	22
2	Laptop	40,000	B	22
3	TV	30,000	B	22
1	Mobile	20,000	C	21
2	Laptop	40,000	C	21
3	TV	30,000	C	21

Self Join

Self Join Apply on same table
q+'h Join it'self.
यह केवल एक ही Table में होता है

id	Name	Age	manager-id
1	A	20	5
2	B	22	3
3	C	23	1
4	D	25	6
5	E	18	2
6	F	19	4

EMP

Select A. Name as Employee,
B. Name as Manger

from Emp as A inner Join
Emp as B on A.mgrid = B. id

Employee	manager
A	E
B	C
C	A
D	F
E	B
F	D

Cursor

- Cursor is temporary memory or temporary workstation.
- इसको करते है temporary workstation इसलिए क्योंकि हम जिसमे Table पर जब DML Operation Rbyrow Perform करते है
- Row by Row अगर काम करना है तो सबसे पहले Table को Cursor में store करते है फिर cursor हम कुछ ऐसे method Provide करता है जिसके साथ हम अपने Result set पर कुछ operation Perform करता है

Types of Cursor

1. Implicit Cursor (Automatic)
2. Explicit Cursor (Manually)

Impilicit Cursor

They are also known as default cursor of SQL Server

(c) Relative n :- यहाँ n में हम Positive or Negative value देंगे,

जैसे $n = 5$

$n = -5$ etc

Relative n depend करता है Absolute n पर।

जैसे Absolute n को $n = 300$ है।

तो यह 300 की Row Fetch करेगा।

अब यहाँ से Relative n के $n = 2$ दे दें

तो यह Absolute n के $n = 300 + 2$ करके Add होगा

$n = 300 + 2 = 302$

or 302 की Row देगा।

और जैसे Relative n के $n = -2$ दें

तो यह Absolute n के $n = 300 - 2$

यानी 298 की Row Fetch होगा।

Declare Cursor

(1) declare MyCursor Cursor School For
- Select Query — Cursor name

(2) Open MyCursor

(3) Fetch method from MyCursor

(4) close MyCursor

(5) deallocate MyCursor.

Explicit Cursor

Created by users when Requirement they fetching data Row-by-Row Manner from table.

Method of Cursor

① Next :- depend करेगा पहले वाले method पर। अगर मुझे Next Record देखना है तो Next method use करूंगा

② Prior :- यह depend करता है। इसके पहले वाले method पर। दोनों यह Last use हुए method की Previous Row Return (Fetch) करता है।

Note :- अगर Prior method से पहले First method use किया जाता है तो यह Blank Return करेगा

③ First :- Independ है। यह Table की First Row fetch करेगा।

④ Last :- Independ है यह Last Row देखा

⑤ Absolute n :- माना किसी Table में 5000 Row हैं उसमें से मुझे Specific Record निकालना है यहां n = 1 or 2 or Any Number होगा। जैसे Third Table की Row find करेंगे।

(c) Relative n :- यहाँ n के साथ positive or Negative value देंगे,

जैसे n = 5
n = -5 etc

Relative n depend करता है Absolute n पर।

मान लो Absolute n के n = 300 है।
तो यह 300 की row fetch करेगा
अब यहाँ से Relative n के n = 2 है
तो यह Absolute n के n = 300 के 2 को Add करेगा
n = 300 + 2 = 302
or 302 की row देगा।
और मान लो Relative n के n = -2 है
तो यह Absolute n के n = 300 - 2
मतलब 298 की row fetch करेगा।

Declare Cursor

(1) declare MyCursor Cursor Scroll For
- Select Query — Cursor name

(2) Open MyCursor

(3) Fetch method from MyCursor

(4) close MyCursor

(5) Deallocate MyCursor.

Table ⇒ Employee

id	Name	Age	Salary
1	A	21	23000
2	B	19	18000
3	C	19	22000
4	D	23	13000
5	E	31	15000
6	F	20	22000
7	G	27	22000
8	H	23	17000

```
declare MyCursor Cursor Scroll For
Select * from Employee
Open MyCursor
Fetch prior from MyCursor
Fetch Next from MyCursor
Fetch Absolute 3 from MyCursor
Fetch Relative 1 from MyCursor
Fetch Relative -2 from MyCursor
Fetch Last from MyCursor
Fetch Next from MyCursor
Fetch Relative 1 from MyCursor
Close MyCursor
Deallocate MyCursor
```

O/P

id	Name	Age	Salary

id	Name	Age	Salary
1	A	21	23000

id	Name	Age	Salary
3	C	19	22000

id	Name	Age	Salary
4	D	23	13000

id	Name	Age	Salary
1	A	21	23000

8	H	23	17000

id	Name	Age	Salary

id	Name	Age	Salary

Temporary Table

they are very similier to Actual table यह table कुछ Time period के बाद delete हो जाती है

Temporary Table are Created in TempDB.

Types of Temp. table

1. Local temp. table
2. Global " "

Local temporary table

Syntax (#)

```
Create table #table_Name
(
    id int primary key identity,
    Name varchar(100),
    Age int
)
```

⇒ यह बताता है कि यह Local temporary table है

- insert into #table_Name values(....);
- Select * from #table_Name;

- Local Temp. table को केवल 1 Connection से ही Access किया जा सकता है।
और वह Connection close करते है, वह Table Automatically Remove हो जायेगी।

- We can creat Local Temp. table with same Name in other Connection.

Global temporary table

(##) => Global temp table.

```
Creat table ##table_Name
(  id int Primary key identity,
   Name Varchar(100),
   Age int,
)
```

- Global temp table are access in other Connection.
- जब वह वह Connection close होगा तो उसमे G. table बनाया, उसे वह Global temp. table delete हो जायेगी।

COALESCE () function find on Return the first Non-Null value

Syntax

COALESCE (Null, Null, 'A', Null, 'B');
O/P A ;

Cast ()

एक data type की value को दूसरी data type की value में convert करता है

I.P Cast (23.45 as int);
O/P 23

Select CAST ('2021-05-03' as date time);
O/P 2021-05-03.0000

Convert () ⇒ Same as cast

Syntax

Select convert (int, 23.45);
O/P 23

Convert date time का format set करता है और datatype convert करता है एक से दूसरे में।
2020-11-17 08:59.44.173 ⇒ convert (varchar, joindate, 103)

O/P 17/11/2020

Rank() & dense_Rank()

- Over Clause mandatory.

id	Name	gender	Marks
1	A	M	50
2	B	M	30
3	C	F	60
4	D	M	65
5	E	F	30

Student : Table

Select Name, gender, Marks, rank(), over (order by marks desc) As Rank from Student.

O/P

id	Name	gender	Marks	Rank	
1	D	M	65	1	1
2	C	F	60	2	2
3	A	M	50	3	3
4	B	M	30	4	4
5	E	F	30	4	5
				6	

Also we can use Partition by

Select Name, gender, marks, Rank() over (Partition by gender order by marks desc) as Rank from student

O/P

id	Name	gender	Marks	Rank
1	C	F	60	1
2	E	F	30	2
3	D	M	65	1
4	A	M	50	2
5	B	M	30	3

Partition by gender

DENSE RANK ()

as Rank

Select Name, gender, Marks, dense_rank() over (Partition by gender, Order by marks desc) from Student

id	Name	gender	Marks	Rank
1	C	F	60	1
2	E	F	30	2
3	D	M	65	1
4	A	M	50	2
5	B	M	30	3

as Rank

Select name, gender, marks, dense_Rank () over (order by marks desc) from Student

id	Name	gender	marks	Rank
1	D	M	65	1
2	C	F	60	2
3	A	M	50	3
4	B	M	30	4
5	E	F	30	4
6	F	M	29	5

- the Rank(), dense_Rank() Raw_Number function are use to Retenive an increasing integer Value.

- ALL of there function Requise order by Clause, Partation by Clause is optional.

Row_Number()

- Over Clause Mandatory.

id	Name	age	Gender
1	A	23	M
2	B	21	F
3	C	10	F
4	D	24	M
5	E	26	M

Student

Select *, Row_Number() Over(order by age desc) as Number from Student

O/P

id	Name	Age	Gender	Number
1	E	26	M	1
2	D	24	M	2
3	A	23	M	3
4	B	21	F	4
5	C	10	F	5

Row_Number = Return Row Number Count the Row.

OVER Clause with Partition by

- Select gender, count(*) from table-name group by gender.

इस SQL query में group by के साथ हम वही coloum होंगे जो select के बाद लिखते हैं और उनके बाद Aggregate function use करेंगे।

अगर मैं Select name, age, gender, Count(*) from table-name group by gender.

करेंगे तो यह Error देगा।

जब हम Aggregate और Non-aggregate column साथ use करते हैं तो हम over clause प्रयोग करेंगे।

Over Clause Introduce in SQL Server 2005.

id	name	age	gender
1	A	23	M
2	B	18	F
3	C	19	F
4	D	25	M
5	E	20	M

Student

Select name, age, count(gender) over (Partition by gender) from Student.

id	name	age	
1	B	F	2
2	C	F	2
3	A	M	3
4	D	M	3
5	E	M	3

Having and group by

Group by

मान लो एक Employee की table है जिसमे multiple department है अब हम department के According No of Employee निकालेंगे है तब हम group by का करेंगे।

id	Name	Dep.	Address
1	A	IT	ABC
2	B	HR	XYZ
3	C	Account	BCD
4	D	Finance	EFG
5	E	IT	HIJ

Employee

Condition-1

जो Column हम group by के साथ देंगे वही select के साथ देंगे।

Ex Select dep from Employee group by dep.

O/P

IT
IT
HR
Account
Finance

अब हम Aggregate function ले सकेंगे।

Select dep, Count(*) from Employee group by dep.

IT	2
HR	1
Accom	1
Finance	1

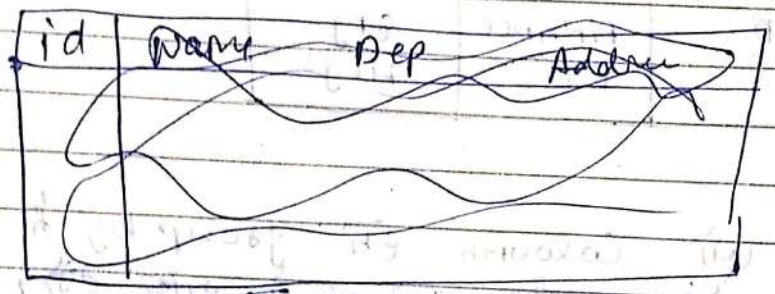
Having Clause

Having Clause को group by Command के साथ use करेंगे। where Clause को use नहीं करेंगे।

E.g.

```
Select dep, Count(*) As Total
dep from Employee, group by
dep Having dep in('IT')
```

O/P



dep	Total dep
IT	2
IT	2

Having Clause	where Clause
① Can be use only Select	① use with select, update, delete
② Filter group	② Filter Rows
③ use aggregate function	③ Not use
④ use with group by	⑤ Can't use with group by
⑤ Can be use after the group by in select query	⑤ Can use it before the group by in select query

Computed Column

वह Column जिसमें हम calculation perform कर सकते है।

```
Create table table_Name
(
  id int Primary key identity,
  Name varchar(100),
  [Basic_Salary] int,
  [House_Allowance] int,
  [Rent_Allowance] int,
  [medical_Allowance] int,
  [Gross Pay] as([Basic_Salary] + [House_Allowance]
  + [Rent_Allowance] + [medical_Allowance])
```

id	Name	Basic_Salary	H.A	R.A	M.A	Gross Pay
1	A	2000	3000	2000	4000	11000

insert into table_Name values ('A', 2000, 3000, 2000, 4000)

Such Column में हम Other Column की value को Calculate कर सकते है।

CTE

Introduce in SQL Server 2005

- CTE एक है Temparoray Result set जिस Result set को हम CTE के द्वारा की हुई Select, Insert, Delete, Update Statment query में use कर सकते है।

Syntax

```
with CTE_Name
as
(
    Select * from table_Name
)
Select * from CTE_Name
```

E.g

id	Name	State	City	Salary
1	A	U.P	SRE	12000
2	B	Gujrat	Rajkot	10000
3	C	U.P	Noida	15000
4	D	U.P.	Noida	22000
5	E	GUJRAT	Surat	13000

Employee

```
with mycte
as (Select * from Employee
    where State = 'U.P'
)
Select City, Count(Id), Sum(Salary) as Salary
from mycte group by City
```

City	Emp_Count	Salary
S.R.E	1	12000
Noida	2	37000

Retering Last generated Identity Valul

Last value or id Get करने के लिए use करते हैं।

There are 3 function to get last generated identity column value in SQL.

1. Scope_identity() : Function
2. @@identity → Global variable
3. ident_current : Function.

Scope_identity()

id	Name
1	A
2	B
3	C

Ref

```
Select Scope_identity();
```

O/P

3

यह केवल current Session (Connection) के According चलता है। New connection के साथ कोई भी value Table में insert करो उसमें कोई फर्क नहीं पड़ेगा। यह उस current session में Table की last generated identity value देगा।

@@identity = यह same work करेगा current connection Only

```
Select @@identity;
```

O/P

3

difference b/w Scope_identity & @@identity

@@identity same connection (session) में different Scope की value Return करता है।

i.e

```
Create trigger trigger_name
  on Raf for insert
  as begin
  insert into cust_detail values (getdate());
  end
```

अब मैं Raf Table कोई value insert करूँगा तो Trigger fire होगा और Result कुछ ऐसा होगा

id	Name
1	A
2	B
3	C
4	D

Same Scope

id	cust date
1	12-05-23 1:00

cust_detail

Another Scope

अब Trigger same session में अलग scope है जो एक cust_detail Table में Date insert कर रहा है। अब जब हम @@identity और Scope_identity() use करेंगे तो result देखेंगे।

```
select Scope_identity()
select @@identity;
```

O/P

4

2

Scope_ident() 4 देगा
or
@@identity 2 देगा क्योंकि यह अलग Scope की value देगा।

ident_current()

ident_current(Table_Name) Return the last identity created for a specific table or view in any session

```
select
ident_current('Raf');
```

O/P

4

Difference

procedure	Function
① may be Return or may Not be Return a value	① always return a value
② ONLy integer value Returns.	② all type value Returns.
③ a Try & Catch block Support	③ Not Support
④ Not get with Select query	④ Get with select query. i:e Select from getFun(s);
⑤ Call function inside Procedure	⑤ Not call Procedure inside function
⑥ Input / output Parameter	⑥ ONLy Input Parameter.
⑦ DML Perform	⑦ NOT Perform DML

Where	Having
① filter the Record from table	① filter Record from group.
② Can not use with group by	② use with group by
③ Can be used with Select, update, delete.	③ ONLy for Select Statment.
④ Where clause will be befor groupby clause	④ Having is used after group by clause.

Order by

① Sort the record in ascending or descding order

② Not Mandatory to use Aggerigate function.

③ SYNTAX dif.

Select * from table_Name order by id ASC;

group by

① group the Record.

② Mandatory to use Aggrigate function.

③ Select city, count(*) from table_Name group by city.

View

① Virtual table

② depend on table

③ does Not Occupie Space ON the System

④ Use to extract the data

⑤ Generate Slow Result because it Render the info from table every time

⑥ delete and Re-Create View No data Loss

⑦ Create View viewName as
- Select quer--

table

① actual table

② Not depend.

③ occupies Space in the System

④ Use to Store the data,

⑤ generate Result fast.

⑥ Re-Create table data will Loss.

⑦ Create table tableName
(
)

View

1) Creat with Select Statement
2) fastest Result
3) Not Parameter Accept
4) Contain Only single query
5) insert into viewname (...)
6) Select * From View_Name

Stored Procedure

1) Create with, Select, insert, update, delete.
2) Slower as compare to view.
3) Accept Parameters.
4) Contain several query.
5) NOT used like view
6) Sp_helptext Sp.Name

Caste ()

1) Syntax:

Cast(Expression, datatype(length)

ie

Cast('2021/5/19', datetime)

Convert()

1) Syntax:

Convert (datatype(length), expression, Style)

Convert (datetime, '2021/5/19', 3);

2) date, time को different-2 format में दिखा सकते है

Rank()

1) Return the Rank
2) if Row value is same then Return the occupie Rank

Marks	Rank	
30	1	1
28	2	2
28	2	3
27	4	4
26	5	5

3) over Clause mandatory

dense_Rank()

1) Return the Rank.
2) Not occupie.

marks	Rank
30	1
28	2
28	2
27	3
26	4

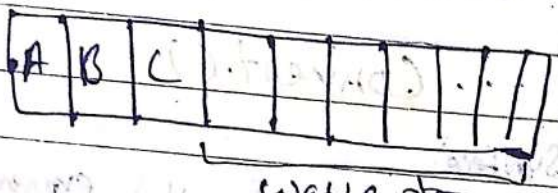
3) over Clause mandatory.

Char, Varchar, nvarchar, nchar diff

Char

@value char(10) = 'ABC'
Print @value
O/P = ABC

यहाँ char यहाँ पर 10 space लेगा लेकिन character length (ABC) मानी 3 है



waste होगा

Print datalength(@value)
Print len(@value)
O/P: 10 | 3

② Fixed length

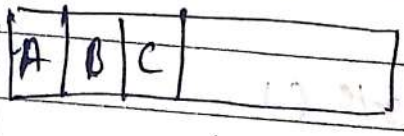
③ 8000 size

④ Char un-unicode है = varchar un-unicode है

Varchar

@value varchar(10) = 'ABC'
Print @value
O/P : ABC

यहाँ varchar memory में space उतना लेगा जितने value के accordingly space लेगा



यह variable length है
NO wastage.

② variable length

③ 8000 size

Note: Simmilier to varchar and char

nchar	nvarchar
① 4000 Size	① 4000 Size
declare @value nchar(4000) = 'ABC'	i.e declare @value nvarchar(4000) = 'ABC'
Print @value	Print @value
Print datalength(@value)	Print datalength(@value)
Print len(@value)	Print len(@value)
O/P	O/P
ABC	ABC
8000 // दो Byte Per Length	6 // यहाँ 6 क्यूँ आया 3 आना था
3	3
	क्योंकि यह एक character के लिए 2 Byte consume करता है।

इन Main difference यह है कि varchar, char 1 byte Per Character Consume करते हैं। पर nchar और nvarchar 2 Byte Per Character क्यों Consume करते हैं।

ऐसा इसलिए क्योंकि nvarchar Unicode Represent करता है। Unicode means Hindi, urdu, malyalam भाषा के characters उन सभी को hold करने की capacity.

IN	EXIST
① Compare the values b/w Sub-query and Parent query	① Can't Compare
② O/P of IN can be True or false or Null	② O/P of EXISTS can be either false or True.
③ Subquery Result is less, then IN is faster than exists.	③ if the subquery Result is Larger, then Exist is faster than IN.

Note:- Normalization is the Process of minimizing redudancy and dependency by organizing fields and table of a database.

Normalization

It is the process of organizing data to minimize data redundancy (data duplication) which leads to data inconsistency.

1 NF :-

- The data in each column should be Atomic, No multiple values seperated by commas (,).

Department_Name	Employee-Name
IT	Suresh, Ravi, Subham
HR	Arjun, Anuj, [redacted]

↓

Not possible to perform query operation on single employee

- No Repeating group

Department-Name	Emp1	Emp2	Emp3	.....
IT	Suresh	Ravi	Subham	[redacted]
HR	Arjun	Anuj	[redacted] space	space

It is not good idea because ~~[redacted]~~ assum that same department can have more than 1000 employee, so we can't create 1000 column.

waste Disk space

2 NF :-

① • the table follow 1 NF.

② • NO Partial dependancy.

Key Key NonKey NON-Key Column

E-id	D-id	Name	Salary	Gender	Dep-Name	Dep-head	Loc-dep
1	1	A	10,000	M	HR	SANTA	Delhi
2	2	B	12000	M	FINANCE	BANTA	Noida
3	1	C	10,000	F	HR	SANTA	Delhi

Partial dependency :- there are E-id and D-id make composite key and identify table Record. but you can see Name, Salary, gender is depend on E-id And Dep-Name, Dep-head Loc is depend on D-id.

NON key column अलग - अलग key column पर depend है जैसे Name, Salary, gender वाले column E-id पर और dep-Name, Dep-head, Loc वाले column D-id पर depend है। इसलिए इन दोनो की अलग Table बनेगी।

E-id	Name	Salary	gender	Dep-id
1	A	10,000	M	1
2	B	12000	M	2
3	C	10,000	F	1

D-id	Dep-Name	head	Loc
1	HR	Santa	Delhi
2	Finance	Banta	Noida

• NON key column एक से ज्यादा key column पर depend नहीं होने चाहिए।

③ Create Relationship b/w there two table. with Primary and foreign key.

3NF:-

- table must be in 1NF and 2NF.
- Should Not have Transitive dependencies

Emp_id	Emp_Name	Gender	Salary	Annual salary	Dept_id
1	A	M	10000	120000	2
2	B	M	12000	144000	5
3	C	F	8000	96000	4
4	D	M	9000	108000	3
5	E	F	15000	180000	1
Key Column	NON Key column	NON Key column	NON Key column	NON Key column	

अब यहा Emp_Name, gender, Salary यह ~~column~~ NON-Key colum, Emp_id (Key column) पर depend है लेकिन Annual_salary वाला column (NON-Key column) Salary वाले Non Key Colum पर depend है यही Transitive dependecie है

इस case में इन दोनो NON Key column को अलग Table बना कर सकेंगे।

Emp_id	Emp_Name	gender	Dept_id
1	A	M	2
2	B	M	5
3	C	F	4
4	D	M	3
5	E	F	1

Salary	Annual Salary
10,000	120000
12000	144000
8000	96000
9000	108000
15000	180000

3rd highest Salary

using max()

id	Name	Salary
1	A	10000
2	B	8000
3	C	13000
4	D	7000
5	E	15000
6	F	18000

Employee

```
Select Max(Salary) from Employee where
    Salary not in
(Select top 2 Salary from Employee
    order by Salary desc)
```

USING MIN().

```
Select min(Salary) from Employee where
    Salary in
(Select TOP 3 Salary from employee
    Order by Salary desc))
```

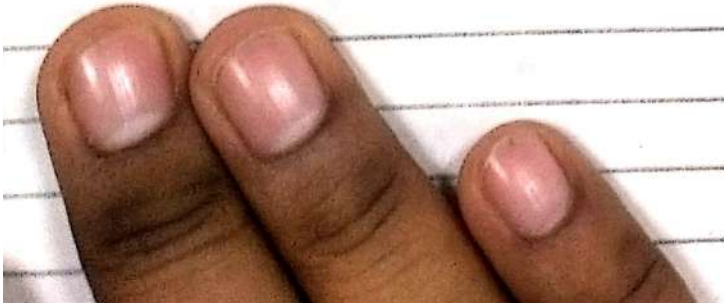
Soo

redundant → अनावश्यक
incorporating → शामिल

Data Integrity :→ It is defines the Accuracy of data and Consistency of data.

De Normalization : It refers to a technique which is used to access data from higher to lower form of database.

Add the redundant data into a table by incorporating database queries that Combine data from various table into a single table.



IN :→ Compare b/w inner query and outer query.

IN :→ Inner or outer query में सबसे पहले Inner query चलेगी।

Select * from Emp where Sal in (Select Sal from Sal)
outer query — एक बार चलेगी — inner query

सबसे पहले inner query चलेगी। उसके O/P को outer query से compare करेगी।

Exists :- सबसे पहले outer query की एक Row को उठायेगा और फिर inner query की हर Row से compare करेंगे।
(Top to down Approach)

Machine test

EMP	Sal
a	50 lc
b	30 lc
c	20 lc